

Lecture I.

1. Reinforcement Learning 涉及多学科. 是一门做出最佳决策的学科. reward System. 神经反射. 运筹学. 博弈论
 RL 不同于监督(非)学习, 其为不断试错的过程(有奖励信号)并得到延迟的奖/惩罚信号. RL 面对的是一个动态过程(还有时间, 但我尚未理解).

2. RL even consider 人类反应时间, 以及 RL 所玩的 game 都 "Roguelike"
 3. RL 是遵循 Reward 假设的模型 (i.e. 所有的目标可被解释为最大化累积奖励)

Goal: 选择行动以得到高回报

以 Time 为度量

一定是动态规划而非贪心.

- C. history $H_t = A_1, O_1, R_1, A_2, O_2, R_2, \dots, A_t, O_t, R_t$ $\rightarrow$ Agent $\rightarrow A_t$
 Agent 是 $H_t \rightarrow A_{t+1}$
 $\leftarrow$ change
 S_t : State: 之前信息的总结

$$S_t = f(H_t)$$

关于 state: 1. S_t^e : 环境 state (部分不可见)

2. S_t^a : Agent state. $S_t^a = f(H_t)$

对未来预测等价

Definition: A state S_t is Markov only if $P(S_{t+1}|S_t) = P(S_{t+1}|S_1, S_2, \dots, S_t)$
 信息不丢失

状态的定义 or 表示 matters.

? = { 以后三个为 state $\xrightarrow{\text{Pred}}$ die

以数量为 state $\xrightarrow{\text{Pred}}$ Cola.

以整个操作 $\rightarrow$?

Light Light open Ring $\rightarrow$ die
 R L O $\bar{O} \rightarrow$ Cola
 O L O R $\rightarrow$?

接下来 我们在 Full observability world. i.e. $O_t = S_t^e = S_t^a$
 这是 Markov decision Process (MDP).

Partial observability: 只有摄像头的机器人不知道地是软的

$S_t^e \neq S_t^a$, 这是 (PO-MDP) (Partial observability).

Agent 需创建 S_t^a 1. $S_t^a = H_t$ 2. $S_t^a = [P(S_t^e = S^1), \dots, P(S_t^e = S^n)]^T$

3. $S_t^a = \sigma(S_{t-1}^a W_s + O_t W_o)$ (σ 是 non-linear 激活函数)

D. inside a RL agent. — component.

mainly: Policy: 模型的行为函数 $S \rightarrow A$

Value function: agent 在某个状态下的好坏

Model: agent 感受的环境

Policy: Deterministic policy: $a = \pi(s)$

Stochastic policy: $\pi(a|s) = P[A=a|S=s]$

Value function: 预测未来的奖励. $V_\pi(s) = E_\pi[R_t + \gamma R_{t+1} + \dots | S_t = s]$

Model: $P_{ss'} = P[S'=s' | S=s, A=a]$ $R = P[R | S=s, A=a]$

状态转移模型

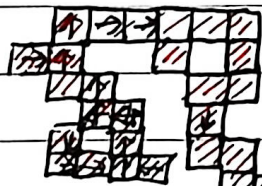
预期的奖励是源于先前的

已知当前环境的 State 与 Action 下

与当下

$\uparrow$ state' 的概率.

e.g. 迷宫. Reward: 时间+1, rewards -1. Action: $\leftarrow \uparrow \rightarrow$ State: 人的位置



Policy: 每个图中的 arrow

Value function: 图中离终点的步数的相反数

Model:  $\rightarrow$ agent 对环境理解.

对 agent 进行分类,

1. Only V: 在迷宫中可, 每个状态有一个 value. 全局 min 即可
2. Only P: 以 some kinds of data structure 表示, 以它得到最大 rewards
3. actor critic. both. V.P.

4. Model Free: no model

5. model based: V.P.M.

E. X

RL:

环境未知: 不断交互, 改变 Policy

Planning:

环境已知: 与 model 交互, 改变 Policy.

Prediction: 未来如何 (按照当前的 Policy). $\rightarrow$ Control (base policy)

我们随机地走, 能有多少奖励 $\rightarrow$ Prediction (注意 Policy 不变)

初为随机走, 发现 value 不行, 改 Policy. $\rightarrow$ Control.

RL 是. 你走了这一步 将来如何.

Lecture 2

MDP 给予了 RL 信息需要的环境

几乎所有 RL 问题可化为 MDP. POMDP 必可化 MDP

状态转移.

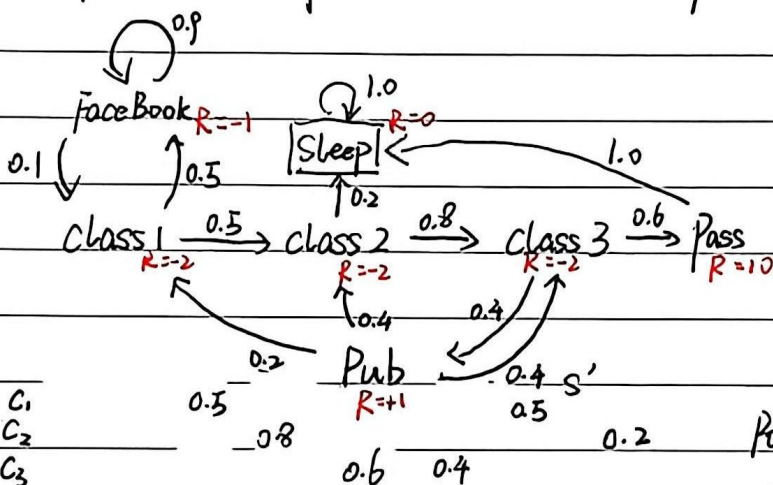
$$P_{ss'} = P[S_{t+1} = s' | S_t = s]$$

Matrix $P = (P_{ij})_{ij}$

Markov process: 一系列有 Markov 性质的 $S_1, S_2, \dots$ (随机状态)

S is a (finite) set of state. Markov process is tuple $\langle S, P \rangle$

e.g.



Markov 过程

$P =$	C_1	C_2	C_3	Pass	Pub	FB	Sleep
C_1							
C_2	0.2	0.4	0.4				
C_3	0.1						
Pass							
Pub							
FB							
Sleep							
S/S'	C_1	C_2	C_3	Pass	Pub	FB	Sleep

Possibility = fit?

不稳定 MDP

Complex Markov

Markov Rewards Process $\langle S, P, R, \gamma \rangle$

+ $R_s = E[R_{t+1} | S_t = s]$, discount factor $\gamma \in [0, 1]$

The return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ future. γ : 对未来的奖励有多爱
 1 sample 从 t 开始, 之后 未来不确定, 防止循环的无限

任何 sample 会收敛

$$V(s) = E[G_t | S_t = s]$$

Bellman Equation.

$$V(s) = E[R_{t+1} + \gamma V(S_{t+1}) | S_t = s] = E[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]$$

$= R_s + \gamma \sum_{s'} P_{ss'} V(s')$ 类似向前传播, 需注意 $V(s)$ 是未来收益

e.g. $V(\text{Class 3}) = -2 + 0.6 V(\text{Pass}) + 0.4 V(\text{Pub})$
 $R = -2$

退出 state 时, 得到这个 state 的 reward

matrices. Bellman Equation. $V = R + \gamma P V$

$$\begin{bmatrix} V(1) \\ \vdots \\ V(n) \end{bmatrix} = \begin{bmatrix} R_1 \\ \vdots \\ R_n \end{bmatrix} + \begin{bmatrix} P_{11} & \dots & P_{1n} \\ \vdots & \ddots & \vdots \\ P_{n1} & \dots & P_{nn} \end{bmatrix} \begin{bmatrix} V(1) \\ \vdots \\ V(n) \end{bmatrix}$$

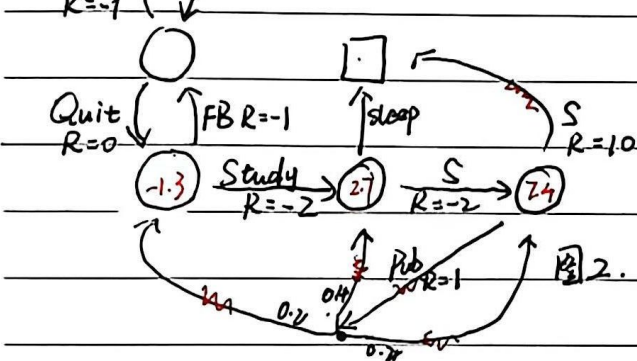
$$V = (I - \gamma P)^{-1} R$$

A Markov decision process ^{MDP} is a tuple $\langle S, A, P, R, \gamma \rangle$

+ A is a finite set of actions

$$P_{ss'}^a = P[S_{t+1}=s' | S_t=s, A_t=a] \quad R_s^a = E[R_{t+1} | S_t=s, A_t=a]$$

$$P_{ss'}^{FB} = P[S_{t+1}=s' | S_t=s, A_t=FB]$$



这里没有概率 每个都是选择 (•除•)

$$\pi(S|A) = 0.5 \quad \gamma = 1$$

→ 下页

$$\text{图 2. } 7.4 = 0.5(1 + 0.2 \times -1.3 + 0.4 \times 2.7 + 0.4 \times 7.4 + 0.5 \times 10)$$

A policy π is a distribution over action given states

$$\pi(a|s) = P[A_t=a | S_t=s] \quad \text{给定 state 下关于 action 的概率分布}$$

Class 1 $\xrightarrow{0.1} FB$
Class 1 $\xrightarrow{0.9} \text{Class 2}$

不在意过去的 rewards 永远注视未来

Policy is time-independent

MDP 总可还原为 MRP 与 MP

给定 MDP $M = \langle S, A, P, R, \gamma \rangle$ and a policy π

1> 遵循某特定流程, 抽取 $S_1, S_2, \dots$ is Markov process $\langle S, P^\pi \rangle$

2> $S_1, R_1, S_2, R_2, \dots$ is a MRP $\langle S, P^\pi, R^\pi, \gamma \rangle$

$$\text{其中: } P_{ss'}^\pi = \sum_{a \in A} \pi(a|s) P_{ss'}^a \quad R_s^\pi = \sum_{a \in A} \pi(a|s) R_s^a$$

消除不确定

$V_{\pi}(s)$ 是按照 π 这个 Policy 决策, reward 的一个评估.

$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$ 这个 state 有多好, 当你选了一个 policy π

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Bellman Equation.

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

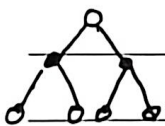
$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

V & q .

$$V_{\pi}(s) = \sum_{a \in A} \pi(a|s) \cdot q_{\pi}(s, a)$$



$$q_{\pi}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{\pi}(s')$$



$$V_{\pi}(s) = \sum_{a \in A} \pi(a|s) \left(R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{\pi}(s') \right)$$

$$V_{\pi} = R^{\pi} + \gamma P^{\pi} V_{\pi} \quad V_{\pi} = (I - \gamma P^{\pi})^{-1} R^{\pi}$$

MDP $\rightarrow$ 环境带你走 MDP $\rightarrow$ 有环境, 你自己有 policy 走

optimal (最优) state-value function $V_{*}(s)$

$$V_{*}(s) = \max_{\pi} V_{\pi}(s). \quad \text{Also, } q_{*}(s, a) = \max_{\pi} q_{\pi}(s, a).$$

$\pi \succ \pi'$ if $V_{\pi}(s) \geq V_{\pi'}(s) \quad \forall s$.

$$\Rightarrow V_{\pi_{*}} = V_{*} \quad q_{\pi_{*}} = q_{*}$$

$$\pi_{*}(a|s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in A} q_{*}(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Bellman Equation.

$$V_{*}(s) = \max_a q_{*}(s, a)$$

$$q_{*}(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{*}(s')$$

Bellman 最优原理.

最优策略的子策略是最优的
(以当前状态为起点的整体)

Bellman Equation 说的是, 问题分为 two parts: 1. act. s.t. 下一个 state 要满足, 对于最优策略 reward 最大. 2. 在新 state 下, 找最优

Lecture 3

Dynamic Programming

问题是序列、时间步的 $\rightarrow$ 优化

DP 将解决复杂问题的子问题

Problems to solve needs to fit two properties:

1. Optimal structure: 最优性原理 最优策略的子策略最优
2. Overlapping subproblems: 子问题不断出现 & 解法可缓存并复用 (递归)

MDP 符合 DP 解决条件

1> Bellman Equation $\rightarrow$ Optimal Structure. (Lecture 2)

2> Value Function $\rightarrow$ Overlapping Subproblem. $V_{\pi}(s)$ 永远告诉你未来最佳要解决的问题

DP 假设 MDP 有所有信息. (Planning not RL)

1> For prediction.

input: $\langle S, A, P, R, \gamma \rangle$ and policy π
or $\langle S, P^{\pi}, R^{\pi}, \gamma \rangle$ ^{MDP} _{MRP}

output: value function V_{π}

2> for Control.

input: $\langle S, A, P, R, \gamma \rangle$ without policy

output: optimal value function V_{*} , optimal policy π_{*}

Policy evaluation.

Problem: evaluate given policy π .

Solution: Bellman expectation Equation. $V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_k$

利用同步备份: 1> At each iteration $k+1$. 2> 对每个 state. 3> 从 $V_k(s') \rightarrow V_{k+1}(s)$

$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) (R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s'))$ 行动得到期望 (环境按概率)



在 terminal 中, $r=0$, 你也出不去.

e.g. 4×4 grid. $\gamma=1$. 阴影为 terminal, 离开网格不影响 value

1	2	3	
4	5	6	7
8	9	10	11
12	13	14	

each step reward = -1. policy π : 随机均匀四方向

$k=0$: V_k 未开始. $\begin{matrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{matrix}$

$k=1$: V_k : $\begin{matrix} 0 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 0 \end{matrix}$ 任何方向移, $r=-1$. 除 shadow.

$\begin{matrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 0 \end{matrix}$

除所有, 在 greedy policy 下. $\leftarrow$

$k=2$: V_k : $\begin{matrix} 0 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 0 \end{matrix}$ 任何走, $r=-1$. 加上上个 V , 中间为 $\frac{1}{4} \sum \pi(a|s) V_k = -2$

$\begin{matrix} 0 & -1 & -1 & -1 \\ -1 & -1 & -1 & 0 \end{matrix}$ $\left[(-1-1) \times 2 + (-1+0) \times 1 \right] \times \frac{1}{3} = -1.7$ Bellman Equation.

$\begin{matrix} 0 & -1.7 & -2 & -2 \\ -1.7 & -2 & -2 & -2 \\ -2 & -2 & -2 & -1.7 \\ -2 & -2 & -1.7 & 0 \end{matrix}$

$\begin{matrix} \leftarrow + + + \\ \uparrow + + + \\ + + + \downarrow \\ + + + \rightarrow \end{matrix}$ $+ = \leftarrow \rightarrow$

$k=3$ 大小关系已确定,

对每点确定了 optimal policy.

V help find π

Policy Iteration

e.g. 左.

Given a policy π , Evaluate it $V_{\pi} = E[R_{t+1} + \gamma R_{t+2} + \dots | S_t = S]$

$\pi' = \text{greedy}(V_{\pi})$, improve π to π'
e.g. 右.

解?

greedy $\rightarrow$ optimise.

EP 迭代 Finally 达到 π^*

Maybe. 小的局部 $\rightarrow \rightarrow \rightarrow \rightarrow$ optimal 整体

e.g. 看课件. important!

new: greedy $\rightarrow$ best action
可以认为是整体贪心
(value 性质).



Consider a deterministic policy π . $a = \pi(s)$

improve. $\pi'(s) = \arg\max_{a \in A} q_{\pi}(s, a)$ 贪心.

$$q_{\pi}(s, \pi'(s)) = \max_{a \in A} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = V_{\pi}(s) \quad \text{value}$$

iterate --- $V_{\pi}(s) \geq V_{\pi'}(s)$

$$q(s, \pi'(s)) = V_{\pi'}(s) = \max_{a \in A} q_{\pi}(s, a) \quad \text{Bellman Equation} \Leftrightarrow \text{convex}$$

$$V_{\pi}(s) = V_{\pi^*}(s) \quad \forall s \in S$$

Value Iteration

Any optimal policy can be subdivide to two parts:

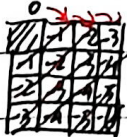
1. Optimal first action A^*

2. 后续状态采用 optimal policy.

假设我们已知 $V_{\pi^*}(s')$

unreachable $\rightarrow P_{ss'} = 0$.

$$\text{Bellman: } V_{\pi^*}(s) \leftarrow \max_{a \in A} R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V_{\pi^*}(s')$$



由于每个格子用最优, (2,2) 就一定要 -2 = -1 + 1/2(-1-1)

information 从后向前传播

Problem: find optimal policy π , Solution: 迭代 Bellman. $V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_{\pi^*}$

迭代的是整个状态空间 S

$$\text{PI: } \pi \xrightarrow{E} V_{\pi} \xrightarrow{I} \pi' \xrightarrow{E} V_{\pi'} \xrightarrow{I} \pi'' \xrightarrow{E} V_{\pi''} \xrightarrow{I} \pi^*$$

$$\text{VI: } V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow \dots \rightarrow V_{\pi^*}$$

V iteration 是一个全state空间波动过程, reward 信息以有限速度传播, 至收敛态
波动到自洽的稳定状态

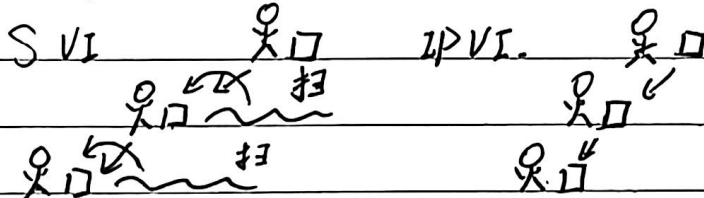
基于 State-value function, $V_{\pi}(s)$. $O(mn^2)$ ^{per iteration} m -action n -state
 action-value function, $q_{\pi}(s,a)$ $O(m^2n^2)$

Prediction Bellman Expectation Equation PE.
 Control BEE + Greedy Policy ^{improvement.} Equation PI.
 Control Bellman Optimality Equation. VI.

DP 扩展: 你是 ^在 $\frac{1}{2}$ per iteration 中知道 every state?

Synchronous VI. $V_k(s') \rightarrow V_{k+1}(s)$

in-place VI. $V(s') \rightarrow V(s)$.



inplace need good order of states.

Bellman error := $\max_{a \in A} (R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a V(s') - V(s))$ guide State selection.

Real-time DP. 在 agent 访问过的 state.

Back up (After each time ^{step} state $S_t, A_t, R_{t+1} \rightarrow S_{t+1}$).

$V(S_t) \leftarrow \max_{a \in A} (R_{S_t}^a + \gamma \sum_{s' \in S} P_{S_t s'}^a V(s'))$

我们用乐观值 $\rightarrow$ 每个区域乐于探索.

这是一个 policy 的总结, $\rightarrow$ 决策态. (PI)

V 是数值计算的中间量.

Lecture 4. Model Free Prediction

L3 we have known MDP evaluate
 L4. estimate V of unknown MDP
 $\rightarrow$ L5. optimise!

Monte-Carlo RL: $\rightarrow$ 多跑几次, 从真实数据中取 mean

从经验片段中学习, 不了解 MDP 中的 reward / transition (model free)
 片段是完整的 (MC 只用 episode 中已知的 G_t , 不同于 $V(s) \leftarrow R_t + V(s')$)

MC 中: value = mean return. $\rightarrow$ 片段必有 terminal
 important. MC can only used in episodic MDPs

Goal: Learning V_π from episodes of experience of under $\pi \rightarrow$ 给定.

$\sim$: 由 π 生成 $S_1, A_1, R_1, \dots, S_K \sim \pi \rightarrow$ 这一条为你已得到的.

MC 的 evaluation 用经验均值而非期望的 return.

First-visit MC Policy Evaluation.

对一个 state 的信息只来源于第一次访问时.

i.e. 在 S 被访问的 ^{in episode} 第一个时间步 t 时间 VS 信息论的未来

计数器 $N(s)++$ Total return $S(s)+ = G(t)$ $V(s) = S(s)/N(s)$

由大数定理: $V(s) \rightarrow V_\pi(s)$ as $N(s) \rightarrow \infty$

Also, 有 Every-visit MC PE.

对于一个 Policy, 在加入多个 episodes 后, MC 得到每个 state 下的 $V(s)$
 对 value function 的影响很多, 但是不需要告诉 agent, 只需一些 returns
 即可得到对结果

Incremental Mean:

$\mu_1, \mu_2, \dots$ 为 $x_1, x_2, \dots$ 的 mean. 有如下递推.

$$\mu_k = \mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1})$$

μ_k 是看过前 $k-1$ 个样本之后, 对

x_k 理论接近 μ_{k-1} "真实 mean" 最好估计

$\rightarrow$ 不是? $\frac{1}{k}(x_k - \mu_{k-1}) \rightarrow$ 新证据与我当前信念的冲突

Incremental MC Update. episode. $S_1, A_1, R_1, \dots, S_T$

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} (G_t - V(S_t)) \quad N(S_t)++$$

fixed → non-stationary

如果不需要 old episode 则: $V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$.

这里并非是用了一个中间状态 G_t (MC 中非法) 而是在一个已经跑完的 episode 中, 按时间步迭代

Temporal-Difference Learning

TD updates a guess towards a guess

TD learns from incomplete episodes, by bootstrapping

用过去估计 future.

MC 猜到墙距离: 不需要到墙, 走几步猜一下

Goal: same as MC.

TD(0) - simplest.

用当前的 V 估计下一个 State

Update $V(S_t)$ towards estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

$R_{t+1} + \gamma V(S_{t+1})$ is called the TD target.

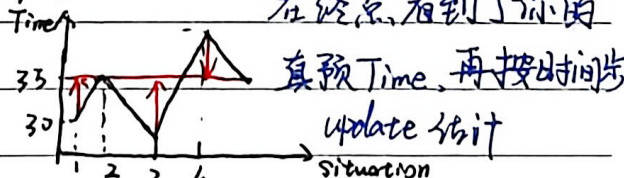
TD error

你在开车, 迎面来了一辆车, 在快撞到时, 该车移开了, MC → 无事故发生.

TD → 差点死了

	已花	预计要多久	总预计
1 出校门	0	30	30
2 与同学聊天	5	35	40
3 遇狗	20	15	35
4 又遇同学	30	20	50
5 到家	35	0	35

MC:

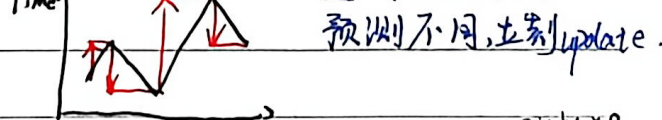


在终点看到了你的

真预 Time, 再按时间步

update 估计

TD:



遇个 situation

预测不同, 立刻 update.

outcome

TD can learn before knowing the final outcome or without the final

$$I(x) = \begin{cases} 1 & x \text{ 成立} \\ 0 & x \text{ 不成立} \end{cases}$$

Bias/Variance Trade-off:

Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ 是 $V_t(S_t)$ 的无偏估计.

True TD target $R_{t+1} + \gamma V_t(S_{t+1})$ 也是 $V_t(S_t)$ 的 $\sim$, 而 $R_t + \gamma V(S_{t+1})$ 为有偏估计.
 $\uparrow$ 偶然, 为真 value function.

TD 的方差要低于 Return. $\rightarrow$ 多步随机
 $\downarrow$ random 一步 + 对未来用某一个定 value func 估计
 MC TD (efficient)
 易收敛 TD(0) 收敛于 $V_t(S_t)$
 对初值不敏感 对初值敏感
 有函数近似 有时无函数近似
 Sample backup $\downarrow$ Full backup $\uparrow$ DP
 Shallow backup $\downarrow$ TD(0)
 Bootstrapping deep backup $\rightarrow$ MC
 MC and TD 最终均收敛于 $V_t(S_t)$, 但当只有有限经验的一批解.

$$S_1^1, a_1^1, r_1^1, \dots, S_T^1$$

$$S_1^k, a_1^k, r_1^k, \dots, S_T^k$$

Two State A, B, 无折扣因子, 8 段. (a 可无)

A0 B0 B1 B1 B1 B1 B1 B1 B0 $V(A), V(B)$.

MC (First time): $V(A) = 0, V(B) = 0$ MC (every time): $V(A) = \frac{0}{1} = 0, V(B) = \frac{6}{8} = \frac{3}{4}$

TD: A $\xrightarrow[r=0]{100\%}$ B $\xrightarrow[r=0.75]{75\%}$ $\rightarrow V(A) = V(B) = 0.75$ Fit MDP, Solve MDP

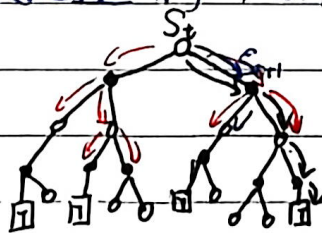
MC: 收敛的解最小均方差 $\sum_{k=1}^K \sum_{t=1}^T (G_t^k - V(S_t^k))^2$

TD(0): 收敛于最大似然 Markov 模型.

$$\hat{P}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^T I(s_t^k = s, a_t^k = a, s_{t+1}^k = s')$$

$$\hat{R}_s^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^T I(s_t^k = s, a_t^k = a) r_t^k$$

也故 TD 更且更适用于 Markov environment



$$MC: V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

$$TD: V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

$$DP: V(S_t) \leftarrow E_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$

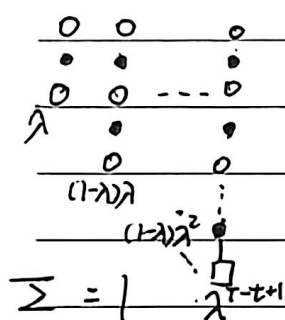
TD(λ): look λ step in future. $\lim_{\lambda \rightarrow \infty} TD(\lambda) = MC$

Define n -step return $G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t^{(n)} - V(S_t))$$

$$\lambda\text{-return } G_t^{(\lambda)} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t^{(\lambda)} - V(S_t))$$



Backward View TD(λ)

Eligibility Trace.

Bell Bell Bell light hit $\rightarrow$ who leads to hitting

Frequency heuristic: assign credit to most frequent states

Recency heuristic: assign credit to most recent states

ET combine both of them.

有人认为Bell影响大, $\mathbb{1}(S_t=S)$ 可提高影响

$E_t(S) = 0$ 有人认为light影响大, $\gamma\lambda$ 越来越多

$$E_t(S) = \gamma\lambda E_{t-1}(S) + \mathbb{1}(S_t=S)$$

有操作 直接 + 1. 不然, 指数级减少



$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

δ_t 按 E_t 反向传播给每一个 State

$$V(S) \leftarrow V(S) + \alpha \delta_t E_t(S)$$

个 State

$\lambda = 0 \rightarrow$ 只为当前状态更新

$$\text{When } \lambda = 1, \sum_{t=1}^T \alpha \delta_t E_t(S) = \sum_{t=1}^T \alpha (G_t^{(1)} - V(S_t)) \mathbb{1}(S_t=S)$$

所有的 credit 均被记录 (无折扣). 结果来看, 与 MC 一致

Backward TD(0)

TD(λ)

TD(1)

B 和 F 只在 offline 时等价

Forward TD(0)

Forward TD(λ)

MC

online 时只近似

Exact online TD(0)

Exact TD(λ)

Exact TD(1)

通过一些构造使得与 offline F 等价

$\lambda = 0$

$\lambda \in (0, 1)$

$\lambda = 1$